CS771 (Autumn 2026) Mini-Project 1

Deadline: Oct 30 2026, 11:59pm

Next-Day PM2.5 Forecasting and PM2.5 Heatmap Generation

In this project, you will analyze and build predictive models for air-quality using data collected recently from a deployment of low-cost PM2.5 sensors in the state of Bihar (and another one collected more recently from a similar deployment in Kanpur, UP).

Among the required tasks (described in more detail in this document) for this project, you will first clean a raw PM2.5 sensor dataset from Kanpur. The next task will be to forecast next day's noon PM2.5 at 468 stations across Bihar from what is observed today, beat (or honestly fail to beat) two fixed baselines, and turn your forecasts into a PM2.5 "heatmap" of the state and visualize this heatmap.

1 The main task

At **12:00 on day** d (which is our *issue time*, i.e., when the forecast is made), forecast PM2.5 at every station for **12:00 on day** $d+1$: a 24-hour lead, and Report the forecast errors in $\mu g/m^3$(a standard measure of PM2.5 concentration).

- **Information cutoff:** the forecast method may use only readings timestamped at or before the issue time, from any station. Please note that nothing from after the issue time may be used as features, for any scaling of features, or for model fitting for that forecast.
- **Why day-ahead:** It's a standard protocol as public air-quality advisories are issued a day ahead so people can plan. A one-hour-ahead forecast is also sometimes used and is often referred to as a "nowcast"; you will study it in Part 3, but it's not the main task.
- **Baselines:** Your reported results must be shown next to two reference forecasts. The first is a very simple *persistence* baseline: forecast tomorrow at 12:00 = sensor reading today at 12:00. The second is a fixed ridge regression model that you build in Part 1. An improvement only counts if your method beats both baselines under the same split and metric.

2 Data

File	Contents
`bihar_pm25_hourly_jan_nov_2024.csv`	Hourly PM2.5 ($\mu g/m^3$) for 468 stations, 1 Jan 00:00 – 30 Nov 23:00 2024, local time (IST). Wide format: one row per hour (first column `timestamp`), one column per station named by its `sensor_id`
`bihar_stations.csv`	Location of each station: `sensor_id`, `latitude`, `longitude` (same order as the columns of the hourly file)
`bihar_grid_005deg.csv`	3,396 grid points inside Bihar at 0.05° ($\approx$ 5 km) spacing: `grid_id`, `latitude`, `longitude`
`bihar_boundary.csv`	The state outline, for plotting
`kanpur_pm25_raw_jul_aug_2026.csv`	Part 0 only: raw, uncleaned PM2.5 from 76 low-cost sensors in and around Kanpur, 1 Jul – 31 Aug 2026, in wide format on a 15-minute grid, times in UTC

- **Package README.** `README.md` describes the file formats and shows how to load them.
- **December data is withheld as the test set.** The course team keeps 1–30 Dec 2024 as a hidden test set (Part 5), so you will not get it see it when working on this project. We will use it to evaluate your final model.
- **Data gaps were pre-filled** from neighbouring stations upstream, and the file does not mark which values were filled. So sensor readings in the dataset given may not be 100% accurate for the imputed values. Consider this as a limitation of the dataset provided.
- **Distances:** convert to approximate kilometres before computing distances:

$$x = (\text{longitude} - 85.5) \times 100.3, \qquad y = (\text{latitude} - 25.7) \times 110.6.$$

3 Fixed rules

A few rules for this project that all the groups need to follow:

- **Split by time.** *Train:* forecasts whose target falls 1 Jan – 31 Aug. *Validation:* targets 1 Sep – 30 Nov. All model and feature choices are made on validation. For the hidden test set evaluation (Part 5), refit your final choice on all targets up to 30 Nov.
- **No leakage.** Every feature for a forecast issued at day d, 12:00 must be computable from readings up to that moment. Scalers, means and any other statistics are fitted on training rows only. Never shuffle rows into a random train/test split.
- **Allowed models:** The may include any method covered up to lecture 16, using standard libraries such as scikit-learn. However, no neural network based models should be used for this project. Feature engineering and being creative with features is absolutely permitted, including learned transformations such as PCA (or other "non-neural" dimensionality methods) or clustering or any other way you think you can make the best use of the features, as long as they are fitted on training data only (see No leakage).
- **Other datasets:** No external data (no weather or satellite data) usage is allowed.
- **Report:** RMSE and MAE at stations, overall and per validation month (Sep, Oct, Nov).
- **Reproducibility:** Use fixed random seeds; the notebook must run from top to bottom.

4 Clarifications

A few addditional clarifications are also worth making.

- **Training data.** Within the training window you may choose which examples to use (for example other issue hours, a subset of stations, or weighting some periods more heavily). Every training target must fall on or before 31 Aug (on or before 30 Nov for the final refit in Part 5). Please state how many training rows you used and why.
- **Validation set.** All validation results must be computed on the full validation set: every station and every 12:00 target from 1 Sep to 30 Nov. Please don't drop stations or days from validation.
- **Inside `forecast`.** You may compute any features from the `history` you are given (for example a station's average over the last 30 days). You may not refit or update the model's parameters inside `forecast`.

5 Part 0: Cleaning a raw sensor dataset (Kanpur)

The Bihar file used in Parts 1–5 described below was already cleaned and gap-filled upstream (i.e., sanitized before we provided it to you). Real sensor data is usually messier and incomplete. In this part, as a learning exercise, you will sanitize a raw file from a network of 76 low-cost PM2.5 sensors installed in Kanpur (data provided for the period July 2026 to August 2026) and document every decision. Part 0 is self-contained; the split rules above apply to Parts 1–5 described later.

1. **Understand the format.** Each sensor has three columns (`<id>_latitude`, `<id>_longitude`, `<id>_pm25`) on a 15-minute UTC time grid. Report: the number of sensors; how often each one actually reports; whether any sensor moves; and how the sensor ID styles differ in the values they report (resolution, typical level).
2. **Reshape to an hourly long table** with columns `sensor_id`, `timestamp_utc`, `latitude`, `longitude`, `pm25`. For any sensor that reports more often than hourly, average the readings within each hour (label each hour by its start). How different is this from simply keeping the reading at minute :00?
3. **Detect problems and flag them**, rather than silently deleting them. At least: any error codes; any implausible zeros; any "stuck" values (the same value repeated for 24 or more consecutive reported hours); any sensors that are mostly unusable (more than 50% of their readings flagged); and any network outages (hours when fewer than 10% of sensors report). For each diagnosis, report how many readings and which sensors it catches, and justify your threshold. Also identify sensors that barely follow the rest of the network.
4. **Write `kanpur_hourly_clean.csv`** with a `flag` column (e.g. `ok`, `error_code`, `zero`, `stuck`, `dropped_sensor`). Leave missing hours missing: do not fill gaps.
5. **Calibration check.** The two main ID styles may come from different device types. Pair each 10-digit-ID sensor with its nearest 8-character-ID sensor, if one is within 3 km. Fit a correction from one family to the other on July and test it on August. Does it reduce the disagreement between pairs? Explain the result.
6. **Summarise** with a before/after table (sensors, readings, share missing) and the network's average daily cycle in **local time** (IST = UTC + 5:30).

6 Part 1: Required baseline

For this part, each group will implement the following, so results are comparable and data handling can be checked.

1. The **Persistence baseline:** forecast = the station's reading at the issue time.
2. The **Ridge regression baseline** with these 29 features per station, all taken at or before the issue time t (day d, 12:00):
   - the station's 24 hourly readings at $t, t-1\,\text{h}, \ldots, t-23\,\text{h}$ (24 features);
   - the mean of its 10 nearest other stations at t;
   - the mean of those 10 stations over $t-23\,\text{h}\ldots t$;
   - the network mean (all 468 stations) at t;
   - $\sin(2\pi d/365)$ and $\cos(2\pi d/365)$, where d = days since 1 Jan 2024 (0 for 1 Jan).
3. Fit scikit-learn `Ridge(alpha=1.0)` on raw (unscaled) features, pooling all stations and all training targets that have 24 hours of history. Clip negative forecasts to 0.
4. Report validation RMSE and MAE, overall and per month.

Question: compare the two baselines on RMSE *and* on MAE. Does one win on both these metric? Why might a model trained on squared error lose on absolute error?

7 Part 2: Improve the forecast

This part allows you to be creative with how you use features or whcih model you use, but still following some rules thar are stated below.

- **Features:** anything computable from readings up to the issue time. Ideas: a longer history (2–7 days), the same hour on several previous days, trends (today minus yesterday), neighbourhood statistics at several radii, the station's long-run training mean, day of week, the overall pollution level, log-transformed inputs.
- **Models:** any allowed model. Tune hyperparameters on validation only.
- **Ablation:** organise your features into 3–8 *feature groups*, where a group is a set of related features built from one idea that you would add or remove together (e.g. "the station's last 24 hourly readings" is one group, not 24 groups). List each group and its number of features. Then, starting from your final model, remove one group at a time, retrain with the same hyperparameters, and report the change in validation RMSE in a table. Keep only groups that earn their place, and comment where groups overlap in the information they carry.
- **Report:** your final model's validation RMSE and MAE overall and per month, next to both baselines.

Questions

- What did each feature group add? Which ideas did *not* help, and why do you think so?
- Did a more flexible model (e.g. a forest) beat ridge on validation? If it did better on the training months but worse on validation, explain what happened.
- What information that you do not have would help most?

8 Part 3: Lead time

Keep the target at 12:00 on day $d+1$ but move the issue time closer: leads of **1, 3, 6, 12 and 24 hours** (issue times 11:00, 09:00, 06:00 and 00:00 on day $d+1$, and 12:00 on day d). For each lead:

1. Persistence: forecast = the reading at the issue time.
2. The Part 1 ridge model, with all features taken relative to the new issue time (the same 24-hour window, ending at the issue time), refitted for that lead.
3. Your Part 2 model, if it can be adapted.

Plot validation RMSE against lead time for all methods, and report the 1-hour results separately from the day-ahead results.

Questions

- Do you see any difference in the behavior of Persistence baseline at 1 h and at 24 h and in between? If yes, what do you think might be the reason (you may consider plotting the average PM2.5 by hour of day.)
- Do the learned models also display the same behavior?
- Which lead time would you choose for a public warning service, and what does that choice cost in accuracy?

9 Part 4: Forecast maps vs observed maps

Interpolate station values to `bihar_grid_005deg.csv` with inverse-distance weighting (power 2, all stations). Use **the same interpolation** for observed values and for forecasts, so map differences come only from the forecasts.

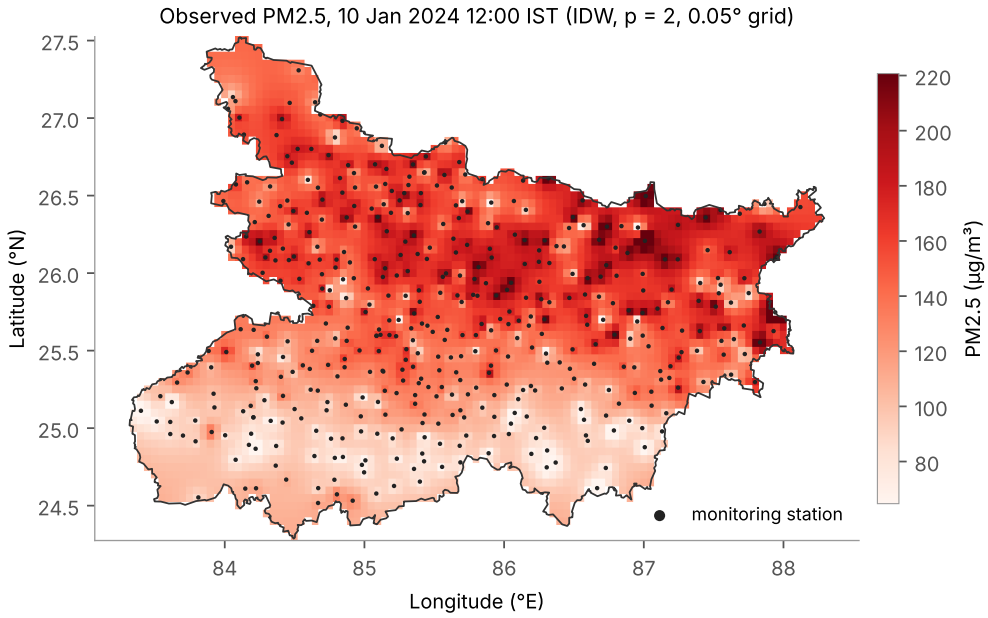


Figure 1: Example of the expected map format (training period, observed values only): PM2.5 at 12:00 IST on 10 Jan 2024, interpolated with inverse-distance weighting ($p = 2$) based interpolation on the 0.05° grid. Your maps should include the same elements: the state outline, the stations, a colour scale with units, and a title giving the date and time.

1. Pick three validation days: one clean, one polluted, and the one where your model's station RMSE was worst. For each, plot side by side: the observed map, your forecast map, the persistence map, and the map of (your forecast − observed). Draw stations and the state outline on top, with one shared colour scale and units.
2. For every validation day, compute: (a) **map RMSE**, the grid-cell difference between forecast and observed maps; (b) **pattern correlation**, the correlation across grid cells between the two maps. Report the averages for persistence and for your model, next to their station RMSE.

Questions

- Which RMSE is better - the map RMSE or for the station RMSE? Which of the two should you quote as forecast accuracy?
- The "observed map" is itself an interpolation. In what sense is it not ground truth?
- Pattern correlation and RMSE can disagree. Describe a forecast that has a good pattern correlation but a poor RMSE. Which matters more for a public warning?

10 Part 5: Hidden December test

Refit your final Part 2 model on all targets up to 30 Nov and submit a file `forecast.py` defining exactly this function:

```
def forecast(history: pd.DataFrame, issue_time: pd.Timestamp) -> pd.Series:
    """history: wide hourly table in the same layout as
    bihar_pm25_hourly_jan_nov_2024.csv (DatetimeIndex 'timestamp', one
    string column per sensor_id), every hour up to and including issue_time.
    issue_time: 12:00 (IST) on some day.
    Returns predicted PM2.5 for issue_time + 24 h, indexed by sensor_id
    (all 468 stations)."""
```

- Start from `starter/forecast_template.py`. Load any fitted model from files you submit alongside (e.g. `model.pkl`); do not refit inside `forecast`. Station coordinates are not passed in: if you need them, include `bihar_stations.csv` in your submission folder.
- Before submitting, run `python starter/check_submission.py <your folder>`. It calls `forecast` exactly as we will be doing during evaluations (on November instead of December) and checks the output format and the time limit.
- The course team will call `forecast` for every issue time from 30 Nov 12:00 to 29 Dec 12:00 (30 forecasts) and score them against the withheld December data.
- Each call must finish within 10 seconds and use only the `history` it is given.
- If your best model on validation is the Part 1 ridge, submit that. Beating ridge is not required for full marks.

You cannot see the December score before submitting, so you cannot tune on it, by design.

11 Use of AI tools

You may use AI tools (e.g. chatbots or coding assistants) for any part of this assignment, but every use must be declared.

Why we allow it. In today's times, AI tools are increasingly used in data analysis and learning to use them effectively can be a useful skill (but not a substitute to the basic understanding of the underlying concepts). Using them well means checking their output, not just accepting it. While working on this assignment, you would realize that it is mostly about judgement: choosing a fair split, spotting leakage, comparing against baselines and explaining what the results mean. Even though the currently available AI tools can speed up the routine coding, they can't make these judgements reliably for you. They also often make subtle mistakes. Allowing them openly, and asking you to declare how you used them, is better than pretending that they do not exist.

- **Declare it.** End your report with a short **Use of AI tools** section listing which tools you used and for what. For example: explaining a concept, suggesting features, writing or debugging code, making plots, or editing text. You do not need to include prompts, but it should be clear which substantial parts of the work involved AI help.
- **You are responsible for the result.** AI-generated code and text count as your group's work. Check them: AI output can be wrong, and can quietly introduce leakage (e.g. fitting a scaler on all rows).
- **Every member must understand the work.** Every group will have a 10 minutes viva (the via schedule will be announced later) in which all group members should be present. During the viva, any member may be asked to explain, modify or justify any part of the submission, including parts produced with AI help.

12 Deliverables and grading

Your final deliverables should be in a zip file containing one notebook that runs from top to bottom (Part 0 may be a separate notebook); the cleaned Kanpur file; `forecast.py` with its model file(s); and a report of at most 6 pages that answers every question above and ends with a short **Use of AI tools** statement.

Submit it on the following URL: `https://www.dropbox.com/request/vzi2to597w1yl101h5ri`

Component	Points	What earns full marks
Part 0: Kanpur cleaning	15	All problem types found with justified thresholds; correct hourly reshaping; a flagged clean file with no gap-filling; an honest calibration test and explanation
Part 1: Required baseline	15	Persistence and ridge reproduce the reference numbers; RMSE vs MAE question answered
Part 2: Improvement	20	Features and model chosen on validation only; a clear ablation table; honest discussion of what did and did not help
Part 3: Lead time	15	Correct curve for all leads; explanation of the persistence dip and the lead-time choice
Part 4: Maps	15	Clear, comparable maps; map RMSE and pattern correlation; correct answers on what the maps can and cannot show
Part 5: Hidden test	10	5 for a `forecast.py` that runs correctly on December; 5 for beating the persistence baseline on December RMSE
Report and code	10	Clear plots with units, honest limitations, reproducible code, no leakage
Total	100	

Grading rewards a sound protocol and clear reasoning over a low error. A model that does not beat ridge, honestly reported and explained, can still earn full marks in Part 2.